

DSA in Python

From Zero to Interview Ready

This book is written for one specific person: someone who knows Python basics, has never solved a DSA problem before, and wants to crack product-based company interviews like PhonePe, Zomato, Swiggy, Myntra, PayPal, and similar. Every word in this book is written with that goal in mind. Nothing extra. Nothing missing.

What this book covers

Phase	Chapters	Topics	Target Companies
Phase 1 Non-negotiable	1, 2, 3	Arrays, Strings, Big-O Hashmaps, Sets Two Pointers, Sliding Window	All product companies All MNCs
Phase 2 Core patterns	4, 5, 6	Recursion Binary Search Stacks and Queues	PhonePe, Zomato Swiggy, Myntra PayPal
Phase 3 Level up	7, 8	Trees and BST Graphs: BFS and DFS	PayPal, Uber Stronger product cos

Contents

Chapter 0 — Before You Write a Single Line of Code

- Why most beginners get stuck
- How Python actually stores things in memory
- The four Python facts that change everything
- How to read this book and study plan

Chapter 1 — Arrays, Strings and Big-O

- What is an array really
- Big-O: the practical version
- String problems are array problems
- Pattern: spot the two-pointer

Chapter 2 — Hashmaps and Sets

- How hash tables work inside
- The frequency counting pattern
- Python collections module
- Pattern: when to reach for a hashmap

Chapter 3 — Two Pointers and Sliding Window

- The one idea behind both patterns
- Fixed window vs variable window
- Pattern recognition: 10 disguised problems

Chapter 4 — Recursion and the Call Stack

- Stop tracing, start trusting
- Base case and recursive step
- From recursion to memoization

Chapter 5 — Binary Search

- Binary search is not one algorithm
- Search on answer space
- Pattern: can I eliminate half?

Chapter 6 — Stacks and Queues

- Basic operations
- Monotonic stack: the hidden pattern

Chapter 7 — Trees

- Traversals: recursive and iterative
- BST properties
- Path problems

Chapter 8 — Graphs

- BFS and DFS
- Connected components
- Topological sort

Before You Write a Single Line of Code

The foundation chapter. No algorithms here. Just the things every beginner needs to know before DSA makes any sense.

0.1 Why most beginners get stuck

You open LeetCode. You see a problem. You stare at it. Nothing comes. You look at the solution. It makes sense when you read it. But the next problem? Same feeling. This cycle repeats for weeks and you feel like you are not making progress.

Here is what is actually happening. You are not stuck because you are bad at coding. You are stuck because you are missing two things: pattern recognition and problem decomposition. These are skills, not talents. They are trainable. This book trains them.

Most people try to memorize solutions. That does not work because interviewers slightly change problems. What works is learning to see the pattern underneath the problem, regardless of how it is dressed up. Every chapter in this book teaches you that pattern explicitly. Not just the algorithm. The way to spot when to use it.

The real goal of this book

Not to make you memorize 200 solutions. To make you look at a new problem and within 2 minutes think: this looks like a sliding window problem. Or: I need a hashmap here. That instinct is what interviews actually test.

0.2 How Python actually stores things in memory

Before you touch any data structure, you need to understand what Python is doing underneath when you write simple code. Most beginners skip this. Then they write slow code in interviews without knowing why it is slow.

Python lists are not arrays

In languages like C or Java, an array is a fixed block of memory. Every element sits right next to the previous one. Python lists look like arrays but they are not. A Python list stores references to objects, not the objects themselves.

What does this mean practically? When you do `my_list[0]`, Python goes to position 0 and finds a reference, then follows that reference to the actual value. This is slightly slower than a true array but gives Python its flexibility.

Python — what a list actually stores

```
my_list = [10, 20, 30]

# Python stores references, not values directly
# Roughly like this in memory:
# my_list -> [ref_to_10, ref_to_20, ref_to_30]
#           |           |           |
#           10          20          30

# Accessing any element by index is O(1)
# It is always one step: go to position, follow reference
print(my_list[0])  # O(1) - instant regardless of list size
print(my_list[2])  # O(1) - same speed
```

Why this matters in interviews

When an interviewer asks you 'what is the time complexity of accessing `my_list[i]`?' the answer is $O(1)$. This is true for any index. It is not $O(n)$. Knowing why (references stored at fixed positions) means you can reason about new situations instead of memorizing answers.

Appending to a list

When you do `my_list.append(value)`, Python adds one element to the end. Most of the time this is $O(1)$, meaning it takes the same time regardless of list size. But occasionally Python needs to resize the internal storage, which takes longer. On average, `append` is $O(1)$. This is called amortized $O(1)$.

You do not need to understand the resizing math right now. What you need to know: `append` is fast. Inserting at position 0 is slow.

Python — append vs insert at beginning

```
my_list = [1, 2, 3, 4, 5]

my_list.append(6)      # O(1) - fast, adds at end

my_list.insert(0, 0)   # O(n) - slow, shifts everything right
# [0, 1, 2, 3, 4, 5, 6]
# Every existing element had to move one position
```

Searching in a list

This one trips up beginners constantly. When you write 'if x in my_list', Python checks every single element from the start until it finds x or runs out of elements. If the list has 1 million elements and x is the last one, Python checks 1 million times. This is $O(n)$.

Python — list search is slow

```
my_list = [1, 2, 3, ..., 1000000]

if 999999 in my_list:  # O(n) - checks up to 1 million elements
    print('found')

# For large lists, this is painfully slow in interviews
# Solution: use a set instead (covered in Chapter 2)

my_set = set(my_list)

if 999999 in my_set:   # O(1) - instant
    print('found')
```

0.3 The four Python facts that change everything

These are not trivia. Each one affects code you will write in interviews. Read each one, run the code yourself, and make sure you understand the implication before moving on.

Fact 1: 'in' on a list is $O(n)$. 'in' on a set is $O(1)$.

Python example

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
num_set = set(numbers)

# Slow - checks every element
if 7 in numbers:    # O(n)
    print('found')

# Fast - hash lookup
if 7 in num_set:    # O(1)
    print('found')

# Rule: if you check membership many times, convert to set first
```

Implication

Implication: whenever you need to check 'have I seen this before?' in a loop, use a set, not a list. This single change turns $O(n^2)$ solutions into $O(n)$ solutions. You will use this in almost every chapter.

Fact 2: Dictionary lookup is $O(1)$.

Python example

```
scores = {'Alice': 95, 'Bob': 87, 'Charlie': 91}

# This is O(1) regardless of dictionary size
print(scores['Alice'])    # 95

# Checking if key exists is also O(1)
if 'Bob' in scores:        # O(1)
    print(scores['Bob'])

# Dictionaries use hashing internally
# Same reason sets are O(1) for membership
```

Implication

Implication: when you need to count occurrences, store computed results, or map one value to another, a dictionary is almost always the right tool. Chapter 2 is entirely about this.

Fact 3: Slicing creates a copy.

Python example

```
original = [1, 2, 3, 4, 5]

# This creates a NEW list, not a view
sliced = original[1:4]    # [2, 3, 4]

sliced[0] = 99
print(original)    # [1, 2, 3, 4, 5] - unchanged
print(sliced)      # [99, 3, 4]

# Slicing costs O(k) time and O(k) space
# where k is the number of elements sliced
# Do not slice inside tight loops without thinking
```

Implication

Implication: slicing looks innocent but hides $O(n)$ cost. A solution that slices inside a loop is likely $O(n^2)$. Two pointers (Chapter 3) exists specifically to avoid unnecessary slicing.

Fact 4: Python integers have no overflow.

Python example

```
# In C++ or Java, integers overflow at ~2 billion
# In Python, integers grow as large as memory allows

big = 2 ** 1000
print(big)    # A 302-digit number. No error.

# This is convenient for interviews
# You never need to worry about overflow in Python
# Unlike Java/C++ candidates who must handle it
```

Implication

Implication: in Python interviews you get one less thing to worry about. When the interviewer asks 'what about very large numbers?' you can confidently say Python handles arbitrary precision integers natively.

0.4 Python tools you must know before Chapter 1

These are built-in Python features that will appear constantly throughout this book. Spend 30 minutes with each one before moving forward.

enumerate — loop with index and value together

Python — enumerate

```
fruits = ['apple', 'banana', 'cherry']

# Without enumerate (the wrong way)
for i in range(len(fruits)):
    print(i, fruits[i])

# With enumerate (the right way)
for i, fruit in enumerate(fruits):
    print(i, fruit)

# Output: 0 apple / 1 banana / 2 cherry

# You will use enumerate in almost every array problem
```

zip — loop over two lists together

Python — zip

```
names = ['Alice', 'Bob', 'Charlie']
scores = [95, 87, 91]

for name, score in zip(names, scores):
    print(f'{name} scored {score}')

# Alice scored 95
# Bob scored 87
# Charlie scored 91

# zip stops at the shorter list
# Use zip_longest from itertools if you need all elements
```

collections.defaultdict — dictionary with default values

Python — defaultdict

```
from collections import defaultdict

# Regular dict raises KeyError if key missing
# defaultdict gives a default value instead

count = defaultdict(int)  # default value is 0

words = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
for word in words:
    count[word] += 1  # no KeyError even first time

print(dict(count))

# {'cat': 3, 'dog': 2, 'bird': 1}

# You will use this constantly in Chapter 2
```

collections.Counter — count anything instantly

```
from collections import Counter

words = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
count = Counter(words)

print(count)
# Counter({'cat': 3, 'dog': 2, 'bird': 1})

print(count.most_common(2))
# [('cat', 3), ('dog', 2)]

# Counter works on strings too
letter_count = Counter('interview')
# Counter({'i': 2, 'e': 2, 'n': 1, 't': 1, 'r': 1, 'v': 1, 'w': 1})
```

0.5 Your study plan

You have 1 to 2 hours per day. That is enough if you use it correctly. Here is the exact structure that works for your situation.

The daily study structure (non-negotiable)

First 20 minutes: review yesterday's problems without looking at solutions. Just look at the problem and write the approach in plain English.

Next 40 to 60 minutes: new content. Read one section, run every code example yourself, then attempt the practice problems.

Last 15 minutes: write in plain English what pattern you learned today and when you would use it. No code. Just words. This builds pattern recognition.

8-week roadmap

Week	Chapters	Problems to solve	Goal
Week 1-2	Chapter 0 + Chapter 1	22 problems	Arrays and Big-O solid
Week 3	Chapter 2	20 problems	Hashmap instinct built
Week 4	Chapter 3	18 problems	Two pointer and window
Week 5	Chapter 4	14 problems	Recursion comfortable
Week 6	Chapter 5	14 problems	Binary search patterns
Week 7	Chapter 6	13 problems	Stack patterns solid
Week 8	Chapters 7 and 8	33 problems	Trees and graphs intro

One rule that matters more than anything else

Never look at a solution before spending at least 20 minutes genuinely stuck. The discomfort of being stuck is where the learning happens. If you look at solutions immediately, you are not building the skill. You are building the illusion of the skill. These feel identical until you sit in an interview.

0.6 How to read a solved example in this book

Every solved example in this book follows the same structure. Learn this structure now so you know what to expect and why each section exists.

Section	What it contains	Why it matters
Problem statement	Exact problem as an interview practice problem	Practice reading real problems
Brute force solution	The obvious slow solution with always complexity	Always understand the naive approach first
Optimized solution	The faster solution with explanation	What you actually code in interviews
Complexity analysis	Time and space complexity of both solutions	Interviewers always ask this
Interviewer note	What thinking pattern they are testing	Building pattern recognition

When you attempt practice problems, write your solution in this same structure. Brute force first, then optimize. This habit will save you in interviews where thinking out loud and showing your process matters as much as the final solution.

0.7 Chapter 0 checklist

Before moving to Chapter 1, make sure you can answer all of these without looking anything up.

1. What is the time complexity of accessing `my_list[i]` for any `i`?
2. What is the time complexity of `'x' in my_list`? What about `'x' in my_set`?
3. What is the time complexity of `my_list.append(x)`?
4. What is the time complexity of `my_list.insert(0, x)`?
5. When you slice a list like `my_list[2:5]`, what does Python actually do?
6. What does `enumerate` give you that a normal `for` loop does not?
7. What is the difference between `dict` and `defaultdict`?
8. What does `Counter` do and when would you use it?

If you cannot answer any of these

Go back and re-read that section. Run the code. Change the inputs. Break it deliberately and see what error you get. Understanding comes from running code, not from reading it.

End of Chapter 0 — Next: Chapter 1: Arrays, Strings and Big-O